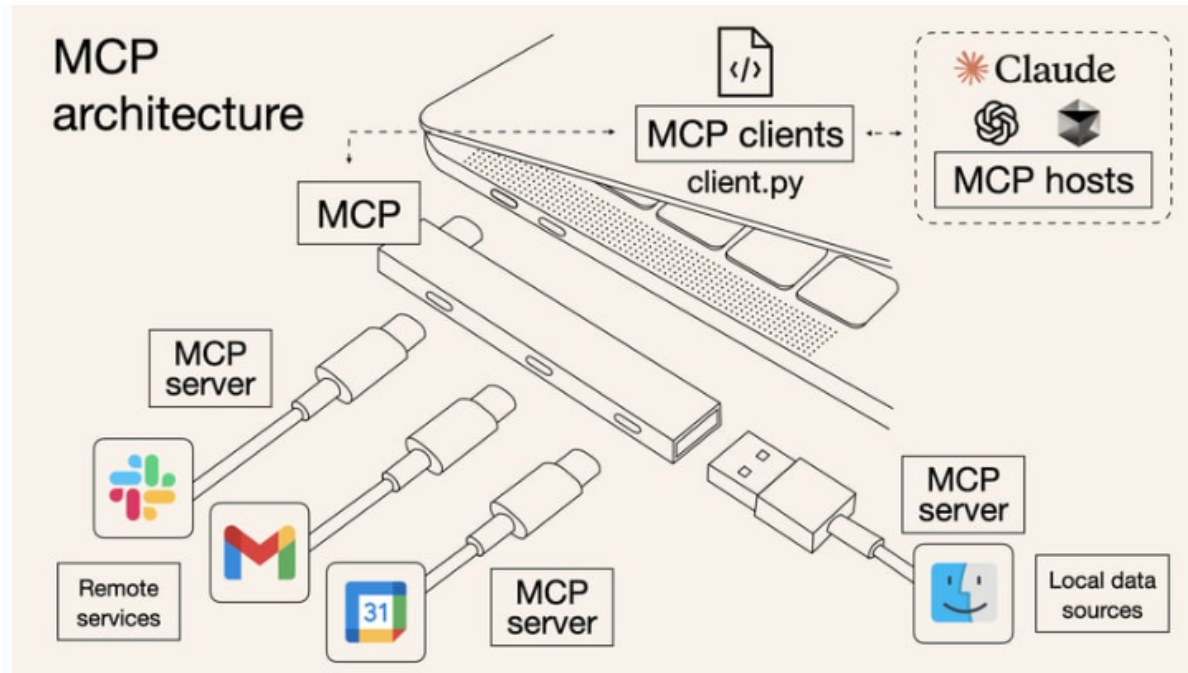


2. Preparation

3. Introduction to the MCP Protocol

- MCP (Model Context Protocol) is an open protocol used to connect large language models with external tools and services. In this system, OpenClaw converts users' natural language instructions into specific robot operation commands through the MCP protocol.
- After the AI model parses the user's instruction, it selects the corresponding MCP Tool to execute. The MCP service converts the tool call into a ROS2 Action command, driving the robot chassis, robotic arm, and other hardware to perform the action.

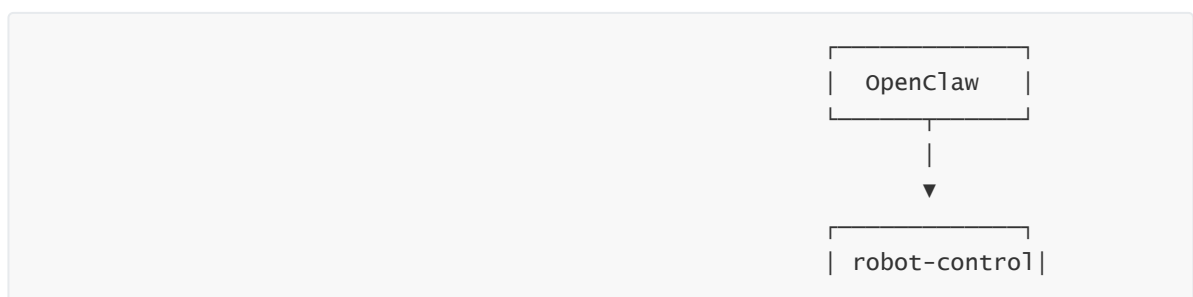


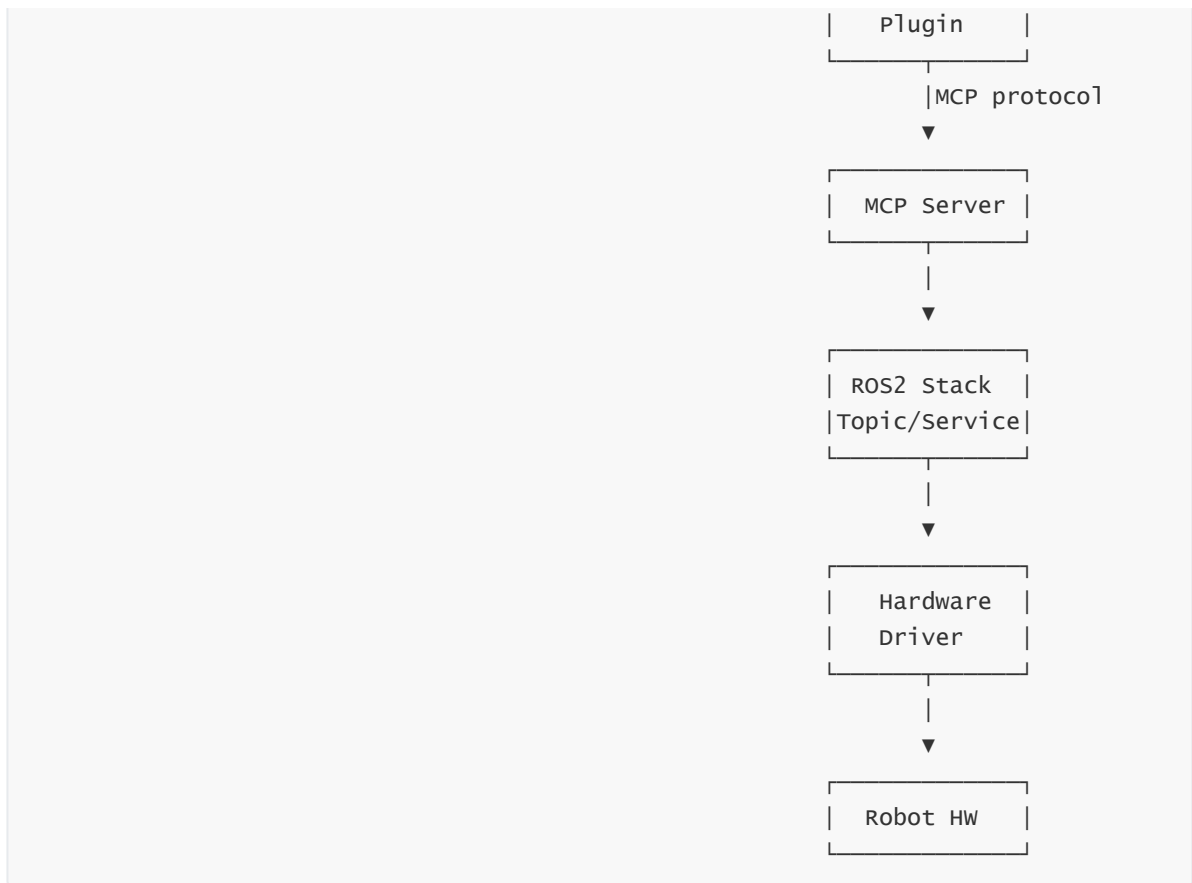
- Through the robot's MCP service interface, the robot can be connected to any AI model, Agent, or AI tool that supports MCP, enabling control of the robot.

4. OpenClaw Robot Control Communication Architecture

The communication architecture for OpenClaw controlling the robot is divided into four layers:

1. **User Interaction Layer:** Send natural language commands through interaction methods such as Web Chat, WeChat, and Feishu
2. **AI Decision Layer:** The LLM in OpenClaw parses the user's intent and selects the corresponding MCP Tool
3. **MCP Service Layer:** Runs on the FastMCP framework, registers tool functions such as chassis movement and visual recognition, and communicates with the robot hardware through ROS2 Action, Topic, and Service
4. **Robot Execution Layer:** The MicroROS chassis agent receives the commands and drives the robot chassis and other hardware to perform actions



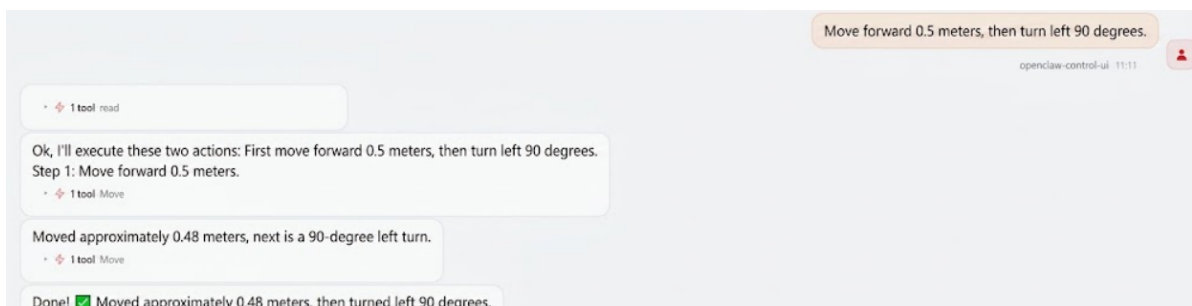


5. Demo Cases

- Control the robot through conversation with OpenClaw
- On a PC that is on the same LAN as the robot's onboard computer, open the Web Chat UI and log in

[!NOTE]

- Any of the interaction methods with OpenClaw can be used; here, the Web Chat interaction is used as an example for the demo
- Here, OpenClaw controlling the robotic arm is used as an example for the demo test:
 - Move forward 0.5 meters, then turn left 90 degrees



5.1 MCP Service

Source Code Path:

```
$HOME/microros_ws/src/multi_brains_pre/multi_brains_pre/mcp_service.py
```

`mcp_service.py` is the entry file of the MCP service, which mainly completes the following work:

1. Create a FastMCP service instance named "ROS-MCP-Service"
2. Create a ROS2 node through `ROSContext` and start background spin
3. Call `register_actions()` to register all robot control tools
4. Start the MCP service and listen on the specified host and port

```
import asyncio
from fastmcp import FastMCP
from .action_controller import ROSContext
import signal
import sys
from .mcp_tools.mcp_action import register_actions

def signal_handler(signum, frame):
    '''Handle system signals for graceful shutdown'''
    ROSContext.shutdown()
    sys.exit(0)

signal.signal(signal.SIGINT, signal_handler)
signal.signal(signal.SIGTERM, signal_handler)

def main(args=None):
    try:
        mcp_service = FastMCP(name="ROS-MCP-Service")
        action_controller = ROSContext().create_node()
        ROSContext.start_spin()

        #-----MCP TOOLS-----
        register_actions(mcp=mcp_service, action_controller=action_controller)

        mcp_service.run(host="0.0.0.0", port=8000, transport="streamable-http")
```

5.2 MCP Tools

Source Code Path:

```
$HOME/microros_ws/src/multi_brains_pre/multi_brains_pre/mcp_tools/mcp_action.py
```

`mcp_action.py` registers MCP Tools through the `register_actions()` function.

All tools follow a unified registration pattern. They use the `@mcp.tool()` decorator to define the description and timeout, use Pydantic Field to constrain the parameter range, and return the `common_response` data class. Take the chassis movement tool **Move** as an example:

```
@mcp.tool(
    description='''Move the robot chassis by a specified distance or rotate
    by a specified angle using odometry feedback. '''
    '''Supported directions: forward, backward, turn_left,
    turn_right. '''
    '''For linear movement (forward/backward), specify distance
    in meters. '''
    '''For rotation (turn_left/turn_right), specify angle in
    degrees (positive value).''',
    timeout=30.0)
def Move(
    direction: Annotated[str, "Movement direction: 'forward', 'backward',
    'turn_left', or 'turn_right'"],
```

```

        distance: Annotated[float, "Linear movement distance in meters. Only
used when direction is 'forward' or 'backward'. Set to 0.0 for rotation."] =
0.0,
        angle: Annotated[float, "Rotation angle in degrees. Only used when
direction is 'turn_left' or 'turn_right'. Set to 0.0 for linear movement."] =
0.0
    )-> common_response:
        res = action_controller.move_by_odom(distance=distance, angle=angle,
direction=direction)
        if res[0]:
            return common_response(execution_result="success", reason=res[1])
        return common_response(execution_result="failed", reason=res[1])

```

5.3 MCP Message Interface

Source Code Path:

```
$HOME/microros_ws/src/multi_brains_pre/multi_brains_pre/message/mcp_message.py
```

`mcp_message.py` uses Python dataclasses to define the request and response message formats of the MCP Tools.

All MCP Tools uniformly return `common_response`. The AI model determines whether the operation succeeded based on the `execution_result` field:

```

@dataclass
class common_response:
    execution_result: str
    reason: str

```